Mini Project Report On

Automated Hostel Administration, Algorithmic Menu Scheduling and Digital Gate Pass System

Submitted in partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science & Engineering

By

Abhijith Sreegith Krishna (U2303004)

Abhiram U (U2303009)

Avinash Vinod (U2303065)

Benito Bitto Palamattathu (U2303071)

Under the guidance of

Ms. Sreejadevi P

Department of Computer Science and Engineering
Rajagiri School of Engineering & Technology (Autonomous)
(Affiliated to APJ Abdul Kalam Technological University)
Rajagiri Valley, Kakkanad, Kochi, 682039

March 2026

CERTIFICATE

This is to certify that the mini project report entitled ***"Automated Hostel Administration, Algorithmic Menu Scheduling and Digital Gate Pass System"*** *is a bonafide record of the work done by* ***Abhijith Sreegith Krishna(U2303004)***, ***Abhiram U (U2303009)***, ***Avinash Vinod (U2303065)***, ***Benito Bitto Palamattathu (U2303071)***, *submitted to the APJ Abdul Kalam Technological University in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B. Tech.) in Computer Science and Engineering during the academic year 2025-2026.*

Ms. Sreejadevi P
Assistant Professor
Dept. of CSE
RSET

Dr. Mary Priya Sebastian
Associate Professor
Dept. of CSE
RSET

Dr. Preetha K.G.
Head of the Department
Dept. of CSE
RSET

ACKNOWLEDGEMENTS

We wish to express our sincere gratitude towards **Rev. Fr. Dr. Jaison Paul Mulerikkal CMI**, Principal of RSET, and **Dr. Preetha K.G.**, Head of the Department of Computer Science and Engineering for providing us with the opportunity to undertake our mini project, "Automated Hostel Administration, Algorithmic Menu Scheduling and Digital Gate Pass System".

We are highly indebted to our project coordinators, **Dr. Mary Priya Sebastian**, Associate Professor, Department of Computer Science and Engineering, and **Ms. Reshma M.R.**, Assistant Professor, Department of Computer Science and Engineering, for their valuable support.

It is indeed our pleasure and a moment of satisfaction for us to express our sincere gratitude to our project guide **Ms. Sreejadevi P** for her patience and all the priceless advice and wisdom she has shared with us.

Last but not the least, we would like to express our sincere gratitude towards all other teachers and friends for their continuous support and constructive ideas.

Abhiram U

Avinash Vinod

Abhijith Sreegith Krishna

Benito Bitto Palamattathu

Abstract

Traditional institutional hostel management relies heavily on fragmented, paper-based processes, leading to severe administrative bottlenecks, security vulnerabilities in student tracking, and financial inefficiencies in dining operations. This project introduces the "Automated Hostel Administration, Algorithmic Menu Scheduling and Digital Gate Pass System," a centralized, AI-driven platform designed to digitize and automate residential facility workflows. To address movement tracking of students, the system implements a Digital Gate Pass, utilizing dynamic Time-Based One-Time Password (TOTP) QR codes that rotate every 10 seconds, effectively eliminating unauthorized proxy entries and physical logbooks. Furthermore, the system revolutionizes mess hall operations through a dual-algorithm intelligence engine; it calculates true dish popularity from student feedback using a Bayesian average, and utilizes a custom Genetic Algorithm to autonomously generate optimized weekly menus that strictly adhere to daily financial constraints. Compared to existing manual methods or disjointed software solutions, this proposed system offers significant advantages: mathematically guaranteed budget compliance, mathematically weighted student feedback, and real-time data synchronization across mobile progressive web apps, administrative dashboards, and IoT lobby signage via persistent WebSocket connections. Ultimately, this architecture transitions hostel management into a secure, highly responsive, and data-driven ecosystem.

Keywords: Hostel Management, Genetic Algorithm, TOTP Security, Bayesian Average, IoT Synchronization.

Contents

Acknowledgements **i**

Abstract **ii**

List of Figures **vi**

List of Tables **vii**

List of Abbreviations **viii**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 2

1.3 Scope and Motivation 2

1.4 Objectives 3

1.5 Challenges 4

1.6 Assumptions 4

1.7 Societal / Industrial Relevance 5

1.8 Organization of the Report 5

2 Software Requirements Specification **7**

2.1 Introduction 7

2.1.1 Overall Description 7

2.2 External Interface Requirements 9

2.3 System Features 9

2.4 Other Nonfunctional Requirements 12

3 System Architecture and Design **13**

3.1 System Overview 13

3.2 Architectural Design 14

3.3 Dataset Identified 14
3.4 Proposed Methodology/Algorithms 15
3.5 User Interface Design 16
3.6 Database Design 30
3.7 Description of Implementation Strategies 32
3.8 Module Division 32
3.9 Work Schedule - Gantt Chart 33

4 Results and Discussions **34**
4.1 Overview 34
4.2 Testing 35
4.3 Quantitative Results 37
4.4 Graphical Analysis 39
4.5 Discussion 40

5 Conclusion **42**
5.1 Conclusion 42
5.2 Future Scope 43

References **44**

List of Figures

3.1 Architecture Diagram 14
3.2 Warden Dashboard: Homepage 16
3.3 Warden Dashboard: Mess reviews 1 16
3.4 Warden Dashboard: Mess reviews 2 17
3.5 Warden Dashboard: Menu management 17
3.6 Warden Dashboard: Menu management - Menu catalog 18
3.7 Warden Dashboard: Menu management - AI menu generation 18
3.8 Warden Dashboard: Overnight logs 19
3.9 Warden Dashboard: Grievances 19
3.10 Warden Dashboard: Resolved grievances 20
3.11 Warden Dashboard: Student management 20
3.12 Warden Dashboard: Student management - Add new student 21
3.13 Student Dashboard: Mess feedback - Veg 22
3.14 Student Dashboard: Mess feedback - Non-veg 22
3.15 Student Dashboard: Mess feedback submission 23
3.16 Student Dashboard: Submit grievance 24
3.17 Student Dashboard: Grievance list 24
3.18 Student Dashboard: Gatepass activity 25
3.19 Student Dashboard: Checkout form 25
3.20 Digital Signage: Default screen with check-in QR 26
3.21 Digital Signage: Mess feedback analytics with check-in QR 26
3.22 WhatsApp BOT: Notice status check 27
3.23 WhatsApp BOT: Alert notice 27
3.24 WhatsApp BOT: Help - User manual 28
3.25 WhatsApp BOT: Set duration for a notice 28
3.26 WhatsApp BOT: Upload notice 29

4.1 Notice upload test 35
4.2 Invalid caption format for notice 35
4.3 Notice without caption 36
4.4 AI Engine test 37
4.5 Menu approval test 37
4.6 Fitness score distribution across an initial high-volume batch of 100 randomly generated menu schedules. 39

List of Tables

2.1 Core System Features of the AI-Driven Hostel Management System 11

3.1 Data Dictionary of Core System Tables 31

3.2 Project Implementation Work Schedule 33

4.1 System Performance and Network Latency Metrics 38

4.2 Genetic Algorithm (Menu Optimization) Evaluation Metrics 39

List of Abbreviations

AI - Artificial Intelligence

API - Application Programming Interface

GA - Genetic Algorithm

GPS - Global Positioning System

IoT - Internet of Things

JSON - JavaScript Object Notation

JWT - JSON Web Token

PWA - Progressive Web App

QR - Quick Response

TOTP - Time-Based One-Time Password

UI - User Interface

Chapter 1

Introduction

1.1 Background

Residential hostels and educational campuses have historically functioned as large-scale accommodation facilities, requiring extensive manual oversight to manage student welfare, facility security, and daily dining operations. Traditionally, these spaces rely on labor-intensive administrative frameworks, utilizing physical ledgers for movement tracking and static corkboards for communication. However, the modern educational landscape is rapidly evolving toward the "Smart Campus" paradigm. With the proliferation of cloud computing and ubiquitous mobile technologies, institutions are increasingly seeking to digitize their infrastructure to improve operational efficiency, reduce administrative overhead, and enhance the overall student living experience.

Despite widespread digital transformation in academic and classroom management, residential facility administration frequently remains technologically stagnant. This persistent reliance on outdated methodologies creates a significant gap between student expectations of a connected, digital-first environment and the operational reality of hostel living. Bridging this gap requires the deployment of modern, data-driven solutions. By integrating real-time synchronization capabilities, cross-platform applications, and algorithmic intelligence into daily administrative workflows, institutions can successfully transition from reactive, manual management to proactive, automated ecosystems.

This paradigm shift provides the foundational context for developing a centralized platform. Currently, one of the primary bottlenecks is that wardens face significant challenges in accurately managing and tracking residents. Student movement and overnight stays are managed predominantly through physical logbooks. This analog approach leaves wardens without real-time data on student whereabouts, making daily tracking, headcounts, and emergency accountability highly stressful and susceptible to proxy entries

and human error. Furthermore, the maintenance of hostel infrastructure relies on highly inefficient physical complaint registers, creating a severe communication gap and delays in resolution.

In addition to security and maintenance, the hostel mess facilities operate on rough manual estimates for food preparation. The calculation of menu costs is performed manually, a practice that frequently leads to severe budget overruns. Beyond financial inefficiencies, the current scenario suffers from a broken student feedback loop. Menus are typically planned based on intuition rather than data, and reviews for individual meals are not systematically collected to gauge true satisfaction. This results in the persistent scheduling of unpopular dishes, leading to high student dissatisfaction.

To address these critical operational gaps, the proposed "Automated Hostel Administration and Management System" operates as a comprehensive, centralized solution. Ultimately, this transitions hostel management from a fragmented, paper-based struggle into a proactive, automated digital ecosystem.

1.2 Problem Definition

Current hostel administration relies heavily on fragmented, manual processes for security, maintenance tracking, communication, and dining operations. This disconnected, paper-based approach creates severe administrative bottlenecks. Specifically, it results in vulnerable access control susceptible to proxy check-ins, inefficient facility grievance management, and massive food wastage due to unpredictable student attendance. Additionally, reliance on static, physical notice boards leads to delayed information dissemination, manual overhead for wardens, and a high likelihood of students missing critical announcements. Furthermore, the lack of a real-time, data-driven feedback loop consistently leads to poor budgetary resource allocation and historically low student satisfaction regarding mess food quality. The core challenge is the complete absence of a centralized, intelligent ecosystem capable of synchronizing these operational workflows.

1.3 Scope and Motivation

The scope of this project encompasses the design and development of a centralized, web-based platform tailored specifically for institutional hostel administration. It integrates

multiple operational modules, including dynamic room allocation, real-time maintenance ticketing, and a secure digital gatepass system. A significant portion of the scope focuses on an AI-driven mess management suite, which features a live Bayesian feedback algorithm and a generative AI engine for autonomous menu planning. The system provides dedicated interfaces for various stakeholders, offering interactive digital signage for students and comprehensive administrative dashboards for wardens and kitchen staff. By focusing strictly on these internal operations, the project delivers a highly specialized ecosystem that directly links resource planning with daily student living metrics.

The motivation behind this project stems from the stark contrast between the rapid technological advancement in education and the archaic, paper-based management of student living facilities. Observing the massive environmental and financial waste caused by kitchens overproducing food—due to a lack of real-time student absence data—highlighted a critical need for integrated systems. Additionally, persistent student dissatisfaction regarding repetitive dining menus and delayed maintenance responses underscored the fundamental failure of traditional, delayed feedback loops. Administrators and wardens are also unnecessarily burdened by manual ledger management, which stifles operational efficiency and proactive decision-making. Ultimately, the drive to apply modern data science and generative AI to these overlooked administrative bottlenecks inspired the creation of a smart ecosystem that prioritizes sustainability, efficiency, and student well-being.

1.4 Objectives

- To develop a centralized, web-based platform that digitizes and streamlines core hostel administrative workflows, including student onboarding, management, and maintenance ticketing.
- To implement a secure Digital Gatepass module that provides real-time tracking of student movement.
- To engineer an intelligent Mess Management suite featuring a generative AI engine capable of autonomously drafting weekly menus and novel recipes that adhere to strict budget, nutritional, and labor constraints.
- To integrate a live Bayesian popularity algorithm via interactive dining hall digital

signage, replacing delayed survey methods with real-time, data-driven feedback on food quality and preferences.

- To create a system for wardens to control the signage screen via WhatsApp.
- To drastically improve overall student satisfaction and living standards by ensuring transparent communication, rapid maintenance response, and a continuously optimized dining experience.

1.5 Challenges

The primary challenges of this project involve architecting a seamless integration between a Python-based generative AI model and a Node.js backend to ensure real-time, autonomous menu creation without system timeouts. Additionally, maintaining strict data consistency across concurrent transactions—specifically synchronizing digital gatepass approvals with dynamic kitchen production metrics—requires highly robust relational database management. Finally, designing an intuitive frontend that accurately captures and processes live Bayesian feedback from interactive student digital signage presents a significant user experience engineering hurdle.

1.6 Assumptions

- It is assumed that the institution has reliable internet connectivity and the necessary hardware infrastructure, such as digital signage and administrative computers, to support real-time data synchronization.
- The project assumes active user adoption, meaning students, wardens, and kitchen staff will consistently utilize the digital gatepass and feedback modules rather than bypassing the system.
- It is assumed that the initial datasets provided for system onboarding—including student registries, dietary constraints, and baseline recipe costs—are accurate and up-to-date.

1.7 Societal / Industrial Relevance

The primary application of this project is deeply rooted in the educational and private student housing sectors, specifically targeting university hostels, boarding schools, and independent Paying Guest (PG) networks. For these institutions, the system provides a centralized digital infrastructure that modernizes outdated, paper-based administrative workflows. By replacing vulnerable physical logbooks with TOTP digital gate passes, the system drastically enhances campus security and ensures reliable student movement tracking for wardens and facility managers.

On a broader societal level, the project directly addresses the critical global issue of food waste. Furthermore, the core architecture has significant industrial relevance beyond educational campuses. The intelligent mess management suite—particularly the live Bayesian feedback engine and the generative AI menu planner—can be seamlessly adapted for corporate cafeterias, hospital dietary wards, military barracks, and large-scale industrial catering facilities that manage fixed populations and require dynamic, cost-effective meal optimization.

1.8 Organization of the Report

The remainder of this report is organized as follows to provide a comprehensive understanding of the project's development and implementation:

Chapter 2: Software Requirements Specification details the core prerequisites of the platform. It covers the overall system description, external interface requirements, specific system features (such as the Warden Dashboard, Student Dashboard and Digital Gate Pass), and other nonfunctional requirements necessary for seamless operation.

Chapter 3: System Architecture and Design outlines the underlying technical foundation of the project. This chapter breaks down the architectural design, identified datasets, and proposed methodologies/algorithms (including the live Bayesian feedback model and Genetic Algorithm menu planner). It also details the user interface design, database design, implementation strategies, and the division of system modules.

Chapter 4: Results and Discussions presents the practical outcomes of the system implementation. It details the testing phases and evaluates the project based on real-world system latency metrics, IoT synchronization speeds, and the quantitative constraint-satisfaction performance of the AI intelligence engine.

Chapter 5: Conclusion and Future Scope summarizes the overall achievements of the multi-platform ecosystem in resolving administrative bottlenecks and optimizing resource management. Finally, it outlines potential future enhancements for the smart-campus infrastructure, including predictive meal demand forecasting and automated room allocation modules.

Chapter 2

Software Requirements Specification

2.1 Introduction

The purpose of this Software Requirements Specification (SRS) is to provide a detailed description of the functional, behavioral, and data specifications for the mini project titled "Automated Hostel Administration, Algorithmic Menu Scheduling and Digital Gate Pass System". This chapter establishes the comprehensive blueprint required to build and deploy the centralized hostel management platform. It meticulously outlines the system's core prerequisites, with a specific focus on the seamless integration between the React-based progressive web application (frontend), the Node.js API server (backend), the Python-based Artificial Intelligence engine, and the IoT security modules.

By clearly defining these critical parameters—including the AI-driven menu optimization utilizing Genetic Algorithms, the real-time digital gate security powered by Time-Based One-Time Password (TOTP) QR codes, and the centralized grievance tracking system—this section serves as the definitive guide for the system's operational scope. It ensures that all stakeholders share a unified understanding of the software's capabilities before advancing into the detailed architectural design and implementation phases.

2.1.1 Overall Description

The "Automated Hostel Administration, Algorithmic Menu Scheduling and Digital Gate Pass System" transitions institutional hostel management into a centralized, three-tier digital ecosystem. It utilizes a hybrid microservices architecture to decouple the user interface, core business logic, and heavy computational tasks, ensuring high scalability and real-time performance.

System Architecture

1. **Presentation Layer:** Students interact via a React-based Progressive Web App (PWA). Wardens utilize a desktop-optimized React administrative dashboard, while an IoT Lobby Signage client serves as a real-time notice board and security terminal.

2. **Logic & Data Layer:** A Node.js backend acts as the API gateway, managing JSON Web Token (JWT) authentication and WebSocket connections (Socket.io). The system employs a chained AI architecture: complex mathematical scheduling and constraint-solving are first offloaded to an isolated Python microservice (Genetic Algorithm). The mathematically optimized output is then piped into the Gemini API via Node.js for creative refinement and natural-language processing. All structured data is stored persistently in a relational MySQL database.

Core Functionalities

1. **Digital Gate Security:** Replaces physical ledgers with Time-Based One-Time Password (TOTP) QR codes that rotate every 10 seconds on the Lobby signage. Students scan these via the PWA to securely authenticate entry/exit.

2. **AI Menu Scheduling:** Employs a two-step AI generation pipeline for weekly planning. First, a Python-based Genetic Algorithm processes daily meal ratings (Bayesian popularity scores) to autonomously generate a baseline menu that strictly adheres to financial and labor constraints. This baseline is then fed into the Gemini API, allowing the Warden to apply natural-language prompts (e.g., "Make Tuesday's dinner more festive") to creatively refine the mathematically sound schedule before final approval.

3. **Centralized Administration:** Includes a digital grievance module for students to report infrastructure issues directly from their phones, and allows wardens to broadcast instant notices to the lobby signage via WhatsApp integration.

User Characteristics

1. **Students:** Require a mobile-first, low-friction interface for quick QR scanning, meal voting, and complaint logging.

2. **Wardens:** Require a high-density, data-rich desktop dashboard to review AI-generated menus, monitor gate logs, and manage maintenance tickets.

2.2 External Interface Requirements

The external interface requirements dictate the precise boundaries across user, hardware, software, and network communication layers to guarantee interoperability within the system's hybrid microservices architecture. From a user interface perspective, the platform delivers a mobile-first Progressive Web App (PWA) built with React.js to facilitate student interactions such as TOTP QR code scanning and grievance logging. Simultaneously, it provides a high-density, desktop-optimized React portal for wardens to manage administrative analytics, alongside a passive, high-visibility IoT Lobby signage interface for broadcasting real-time security tokens. On the hardware front, the system intrinsically relies on the physical camera modules of student smartphones to capture and decode the dynamic QR codes , while the lobby signage necessitates a standard visual output interface, such as HDMI, to continuously render the digital signage. Regarding software and communication interfaces, the React clients interact with the centralized Node.js API gateway exclusively through stateless HTTP/HTTPS RESTful APIs utilizing standardized JSON data payloads. To achieve sub-second, real-time synchronization for the 10-second TOTP rotations and emergency notice broadcasts, the system establishes persistent, bidirectional WebSocket connections via Socket.io between the Node.js server and the IoT signage. Furthermore, the Node.js orchestrator interfaces with an isolated Python environment for intensive AI computations—specifically the menu-generating Genetic Algorithm—by exchanging serialized JSON data through standard input/output streams to prevent main-thread blocking. Finally, the backend executes structured SQL queries to interact with a persistent MySQL relational database and integrates external webhooks for a WhatsApp Bot service, allowing wardens to remotely trigger dynamic signage updates.

2.3 System Features

The system is compartmentalized into four primary functional modules, each designed to address specific administrative and operational bottlenecks within the hostel environ-

ment. These core features operate concurrently to provide a seamless, real-time experience for both students and wardens.

Feature Module	Description	Primary Users
Digital Gate Pass & Security	Replaces written overnight registers with a secure digital check-in/check-out system. Utilizes Time-Based One-Time Password (TOTP) technology to generate dynamic QR codes that rotate every 10 seconds on the lobby display to prevent proxy usage.	Students, Wardens
AI-Driven Menu Optimization	Captures daily student mess reviews to calculate Bayesian meal satisfaction scores. It employs a chained AI pipeline: a Python-based Genetic Algorithm first generates a baseline weekly menu mathematically optimized for budget, labor, and popularity constraints. Subsequently, the integrated Gemini API allows Wardens to creatively refine this schedule using natural-language prompts before final approval.	Wardens, Kitchen Staff
Grievance Ticketing System	Provides a centralized digital platform for students to log categorized facility complaints (e.g., infrastructure, food) with optional image attachments. Wardens can track, assign, and resolve these tickets via a live dashboard.	Students, Wardens
IoT Signage & Notice Broadcasting	A lightweight IoT client in the hostel lobby connects via WebSockets to display the rotating TOTP QR codes and real-time satisfaction graphs. Wardens can remotely broadcast digital notices to this screen via a WhatsApp Bot integration.	All Residents, Wardens

Table 2.1: Core System Features of the AI-Driven Hostel Management System

2.4 Other Nonfunctional Requirements

The system design must satisfy several critical operational, architectural, and nonfunctional constraints to ensure security, reliability, and usability within the hostel environment.

Security and Anti-Spoofing: The digital gate pass QR code displayed on the Lobby signage must use a Time-Based One-Time Password (TOTP) algorithm that rotates every 10 seconds. To prevent unauthorized proxy check-ins, scanned codes must be invalidated immediately upon use, preventing students from utilizing screenshots or sharing access. Furthermore, concurrent database write operations—such as multiple students scanning the gate pass simultaneously—must be handled using secure SQL transactions to prevent data race conditions.

Performance Constraints: The system must deliver near-instantaneous feedback for student movement; the backend response time for successful gate pass check-ins must be less than 1.5 seconds. Additionally, because the Genetic Algorithm used for menu optimization is computationally heavy , the system architecture must ensure that the AI engine completes its processing and returns a valid 7-day schedule within a strict timeout threshold to prevent the frontend dashboard from freezing.

System Availability and Reliability: Because the digital gate pass and grievance modules are critical to daily hostel operations, the core Node.js backend and MySQL database must be designed for high availability. The system must maintain low-latency, persistent WebSocket connections between the backend server and the Lobby signage to ensure real-time updates. In the event of network disruption, the signage must implement robust auto-reconnection logic to recover instantly without manual intervention.

Platform Compatibility: The student-facing application must be implemented as a responsive Progressive Web App (PWA) that runs flawlessly on major mobile browsers such as Chrome, Safari, and Firefox. In contrast, the Warden Dashboard must be optimized for high-density information display on desktop monitors.

Chapter 3

System Architecture and Design

3.1 System Overview

The proposed Automated Hostel Administration System is architected as a three-tier modular web application. The architecture fundamentally decouples the user interface (presentation layer), the core business logic (application layer), and the data storage mechanism to ensure high scalability and maintainability. To handle the varying computational demands of hostel administration, the system employs a Hybrid Microservices-inspired approach featuring a dual-engine AI architecture. Standard CRUD (Create, Read, Update, Delete) operations, JWT authentication, real-time WebSocket connections, and external integrations such as the Gemini API for natural-language, creative menu planning-are orchestrated by a central Node.js backend server. Conversely, computationally intensive tasks—specifically the mathematical algorithmic menu scheduling - are entirely offloaded to an isolated Python service. This isolation ensures that heavy evolutionary algorithm computations do not block the main server thread, guaranteeing a seamless, low-latency user experience across the React-based frontend and IoT display clients.

3.2 Architectural Design

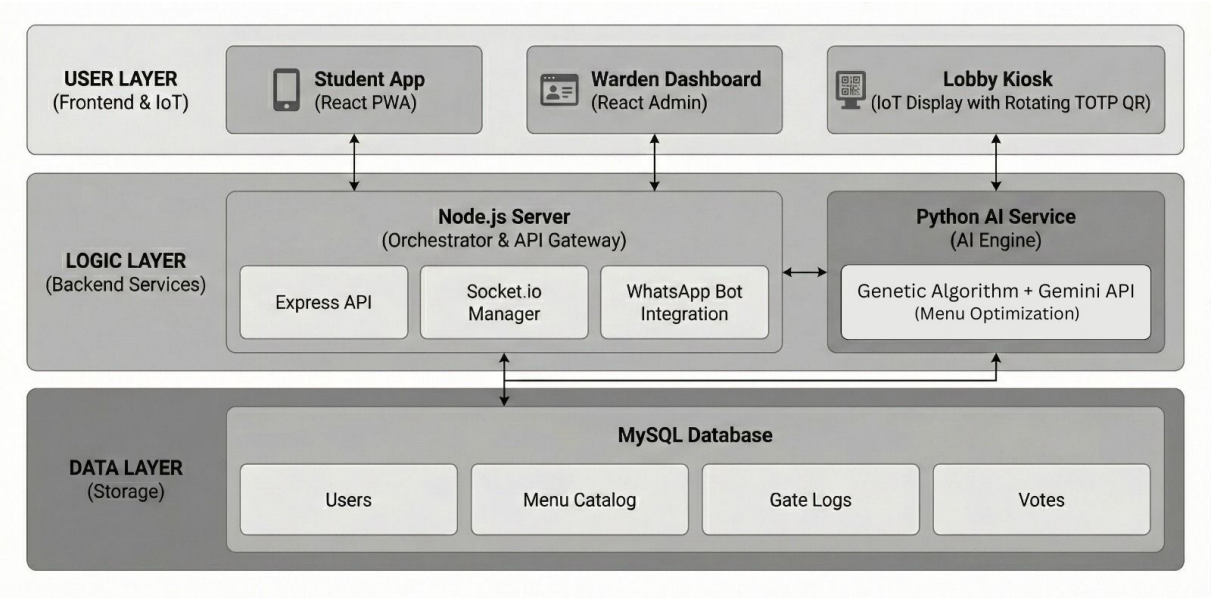

Figure 3.1: Architecture Diagram

3.3 Dataset Identified

The core intelligence of the system relies on two primary datasets: the Menu Catalog and the Mess Reviews. The Menu Catalog serves as the immutable reference dataset for all AI-driven food operations. It contains data for distinct dish combinations, with essential properties including dish id, dish name, effort score, type (Veg/Non-Veg), popularity score and cost per serving. The secondary dataset, Mess Reviews, is dynamically generated by the user base. It captures real-time student feedback, recording attributes such as review id, student uid, the specific meal type, the date served, a numeric rating value (1-5 stars), dish issues, written comments and timestamp. By synthesizing the static cost parameters of the Menu Catalog with the dynamic voting data from the Mess Reviews, the AI engine makes informed, data-driven scheduling decisions.

3.4 Proposed Methodology/Algorithms

The system relies on a dual-algorithm approach to fully automate and optimize mess hall operations.

First, to accurately gauge student satisfaction, the system implements a Bayesian Average Algorithm. Simple arithmetic averages are highly susceptible to skewing (e.g., a dish with a single 5-star review outranking a dish with fifty 4.5-star reviews). To normalize this, the system calculates a weighted popularity score for each dish in the Menu Catalog using the following Bayesian formula:

$$W = \frac{R \cdot v + C \cdot m}{v + m}$$

Where W is the weighted popularity score, R is the arithmetic mean rating for the specific dish, v is the total number of votes the dish has received, m is the minimum number of votes required to be considered statistically significant, and C is the mean rating across the entire menu catalog.

Once the Bayesian popularity scores are calculated, the system utilizes an evolutionary Genetic Algorithm for combinatorial menu optimization. The algorithm is constrained by the strict daily budget limit set by the warden. It initializes a population of "chromosomes," where each chromosome represents a potential 7-day meal schedule. The fitness function evaluates each schedule based on maximizing the total Bayesian popularity score while applying severe mathematical penalties if the sum of the dish costs exceeds the daily budget constraint. The fitness function $F(x)$ is defined as:

$$F(x) = \sum_{i=1}^{7} (P_i - \alpha \cdot \max(0, C_i - B))$$

Where P_i is the popularity of the daily menu, C_i is the calculated cost of the daily menu, B is the strict budget limit, and α is a heavy penalty coefficient for budget violations. Through selection, crossover, and mutation, the algorithm converges on an optimal weekly menu.

3.5 User Interface Design

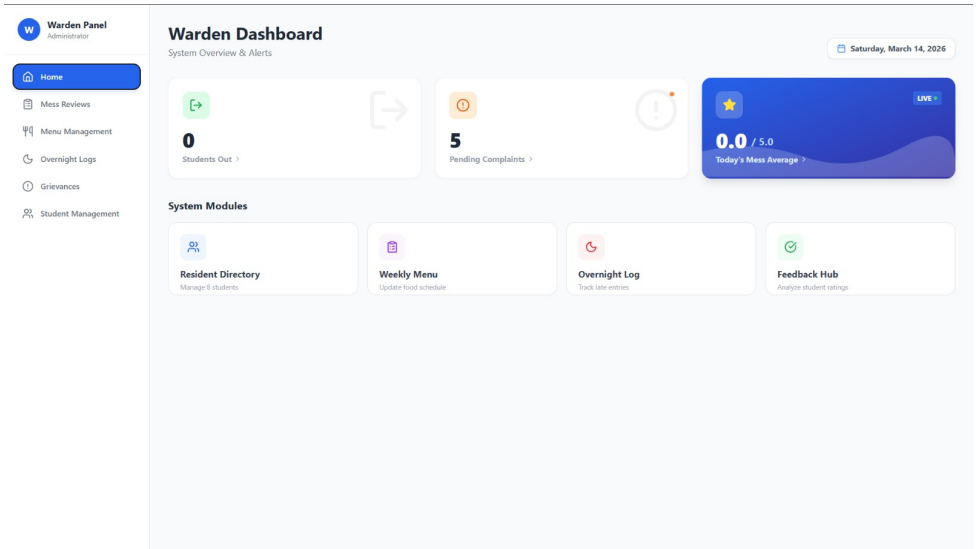

Figure 3.2: Warden Dashboard: Homepage

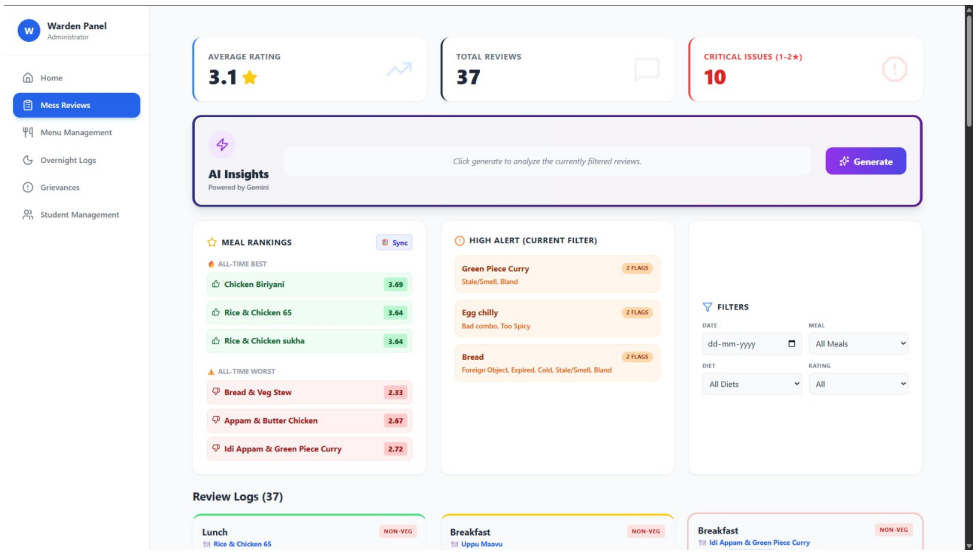

Figure 3.3: Warden Dashboard: Mess reviews 1

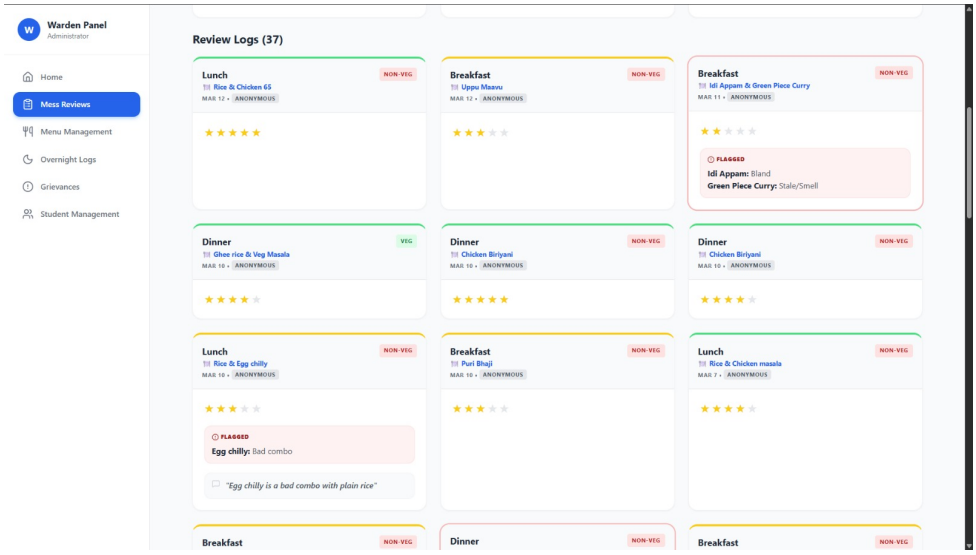

Figure 3.4: Warden Dashboard: Mess reviews 2

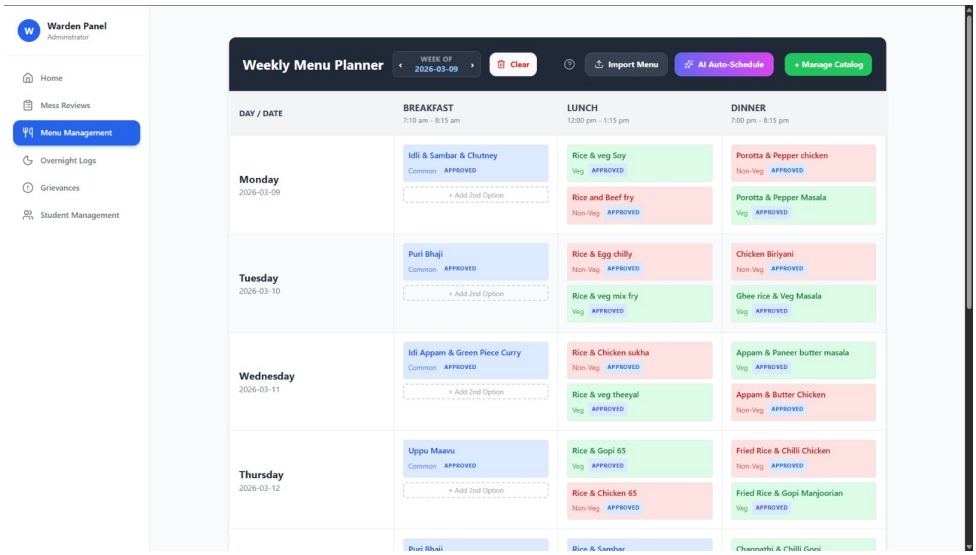

Figure 3.5: Warden Dashboard: Menu management

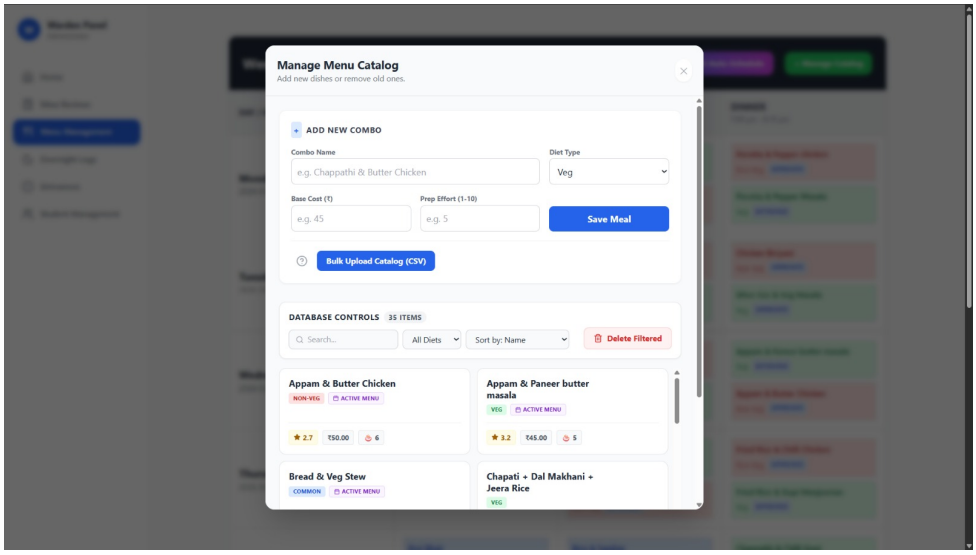

Figure 3.6: Warden Dashboard: Menu management - Menu catalog

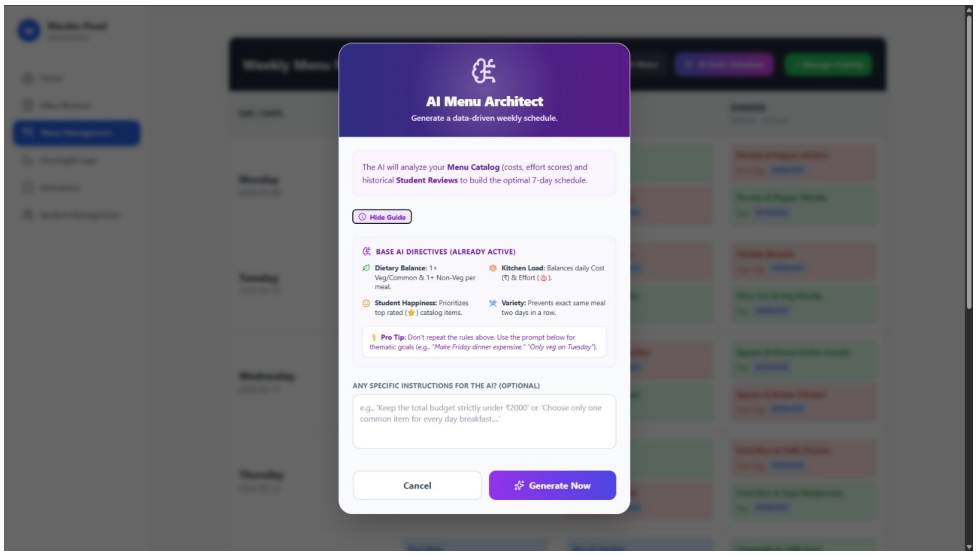

Figure 3.7: Warden Dashboard: Menu management - AI menu generation

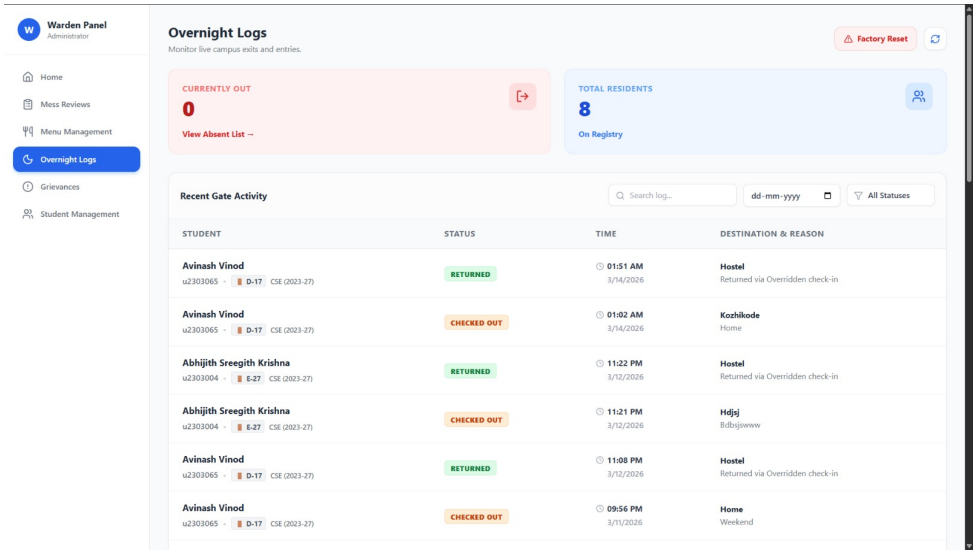

Figure 3.8: Warden Dashboard: Overnight logs

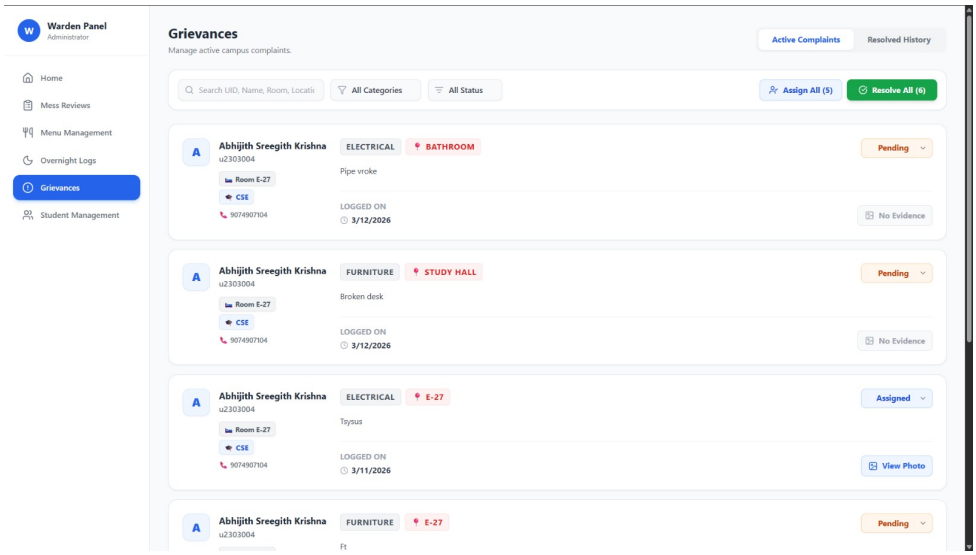

Figure 3.9: Warden Dashboard: Grievances

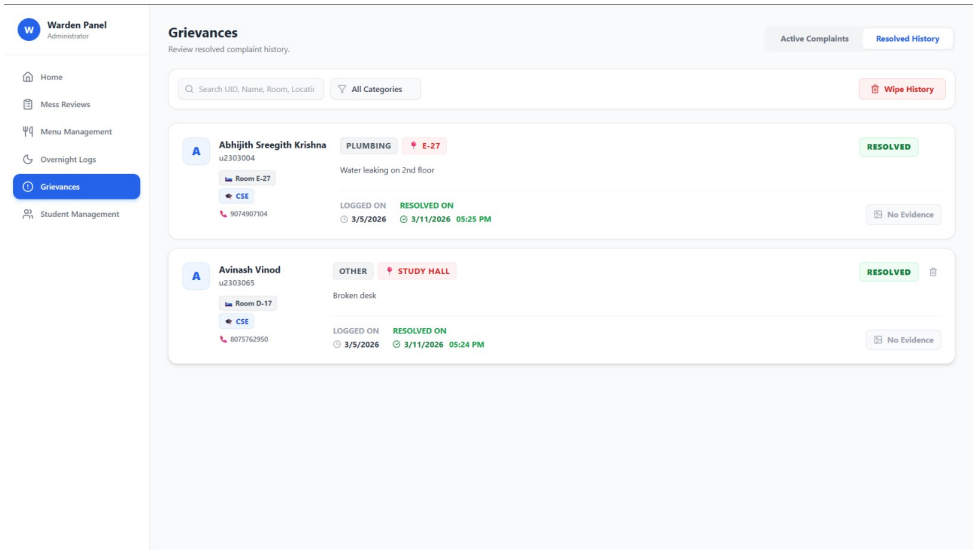

Figure 3.10: Warden Dashboard: Resolved grievances

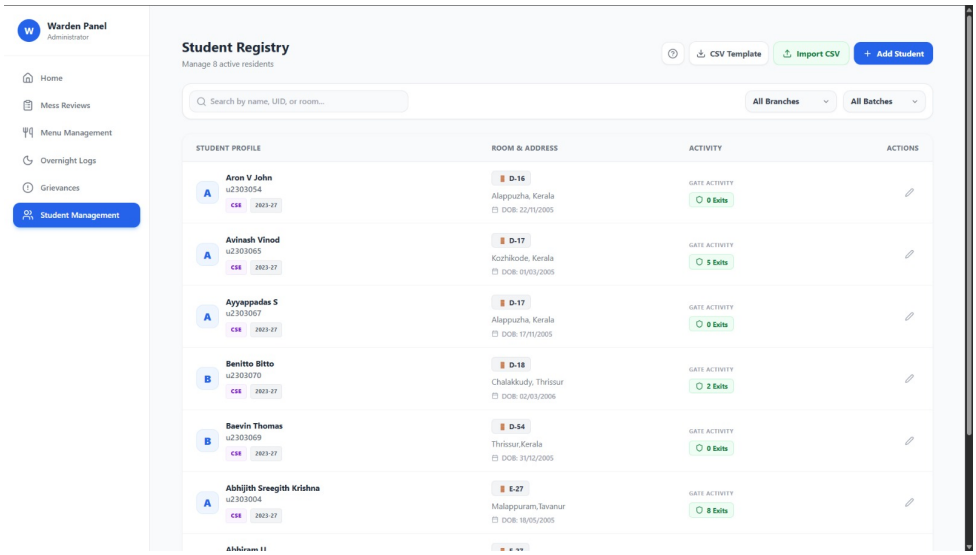

Figure 3.11: Warden Dashboard: Student management

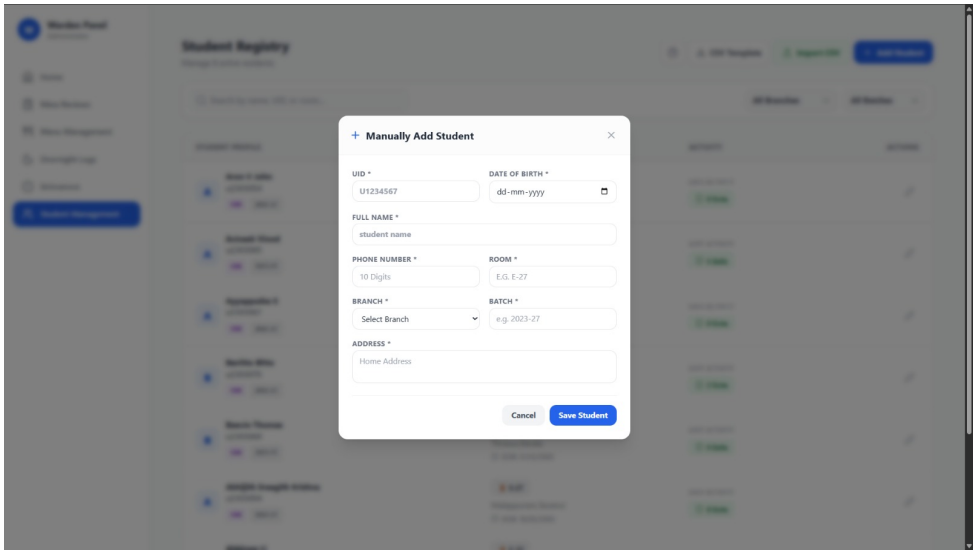

Figure 3.12: Warden Dashboard: Student management - Add new student

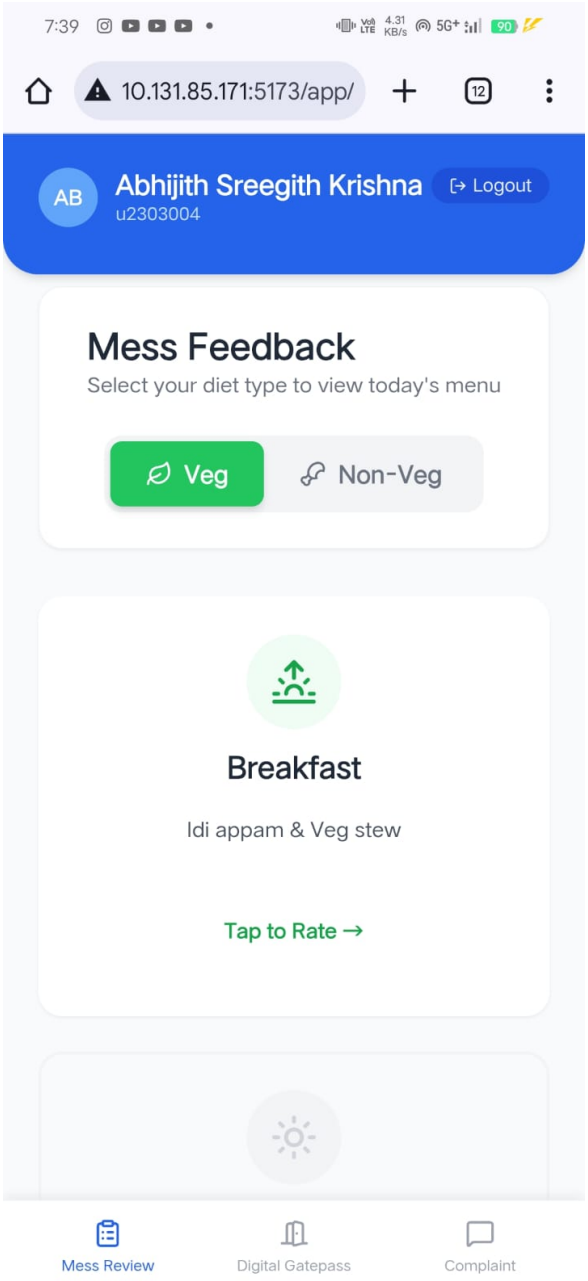

Figure 3.13: Student Dashboard: Mess feedback - Veg

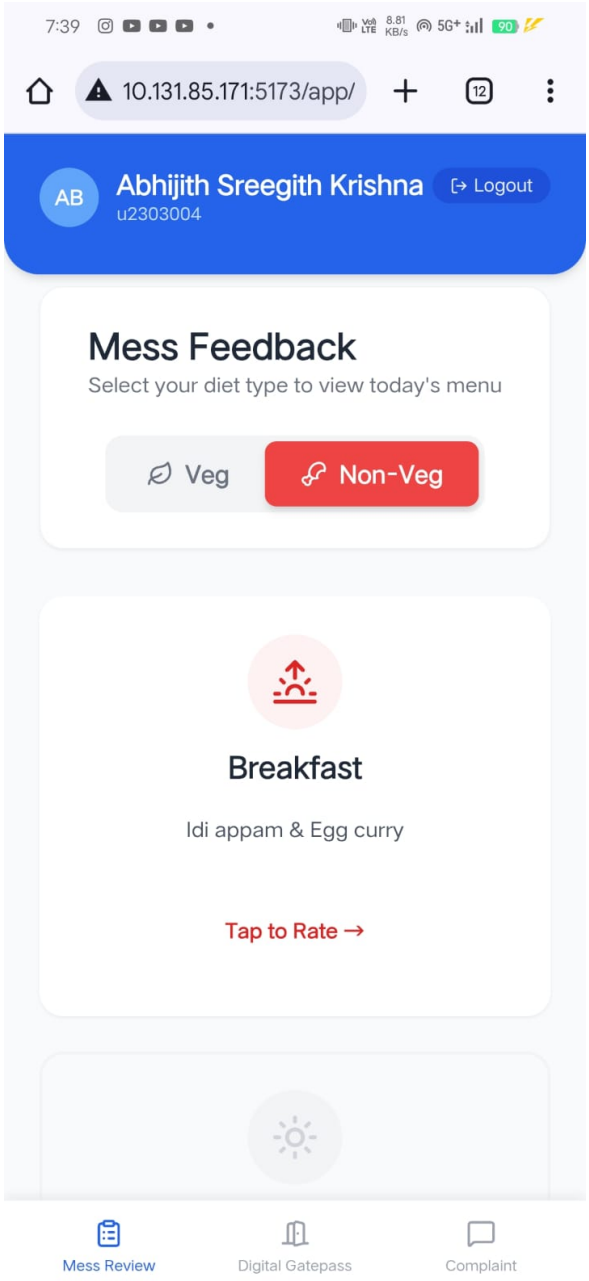

Figure 3.14: Student Dashboard: Mess feedback - Non-veg

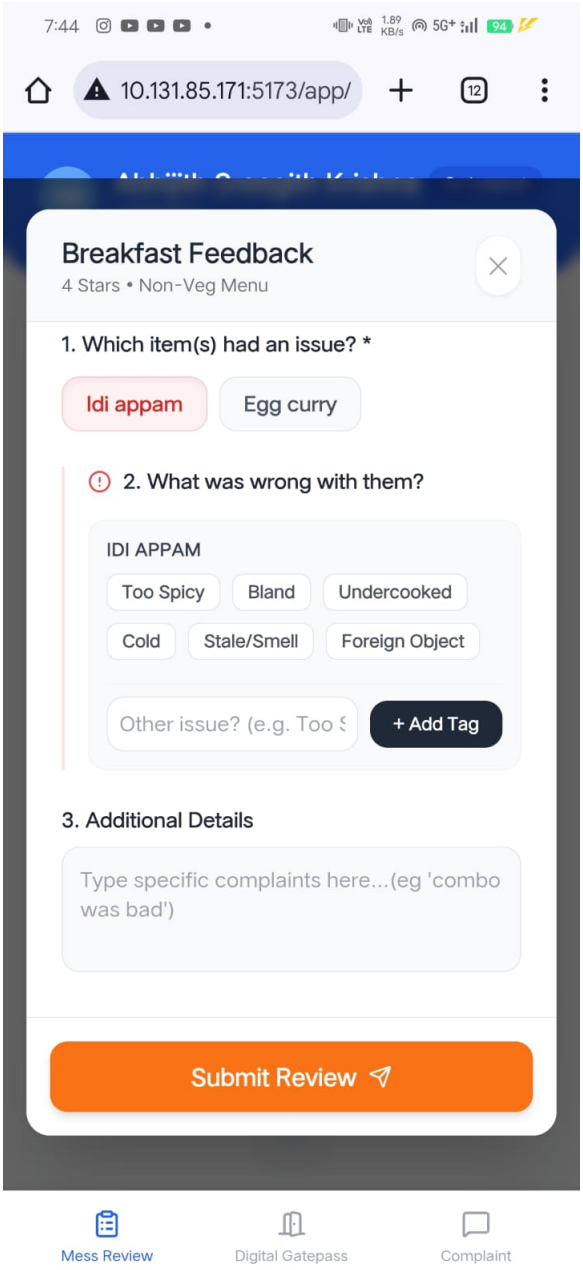

Figure 3.15: Student Dashboard: Mess feedback submission

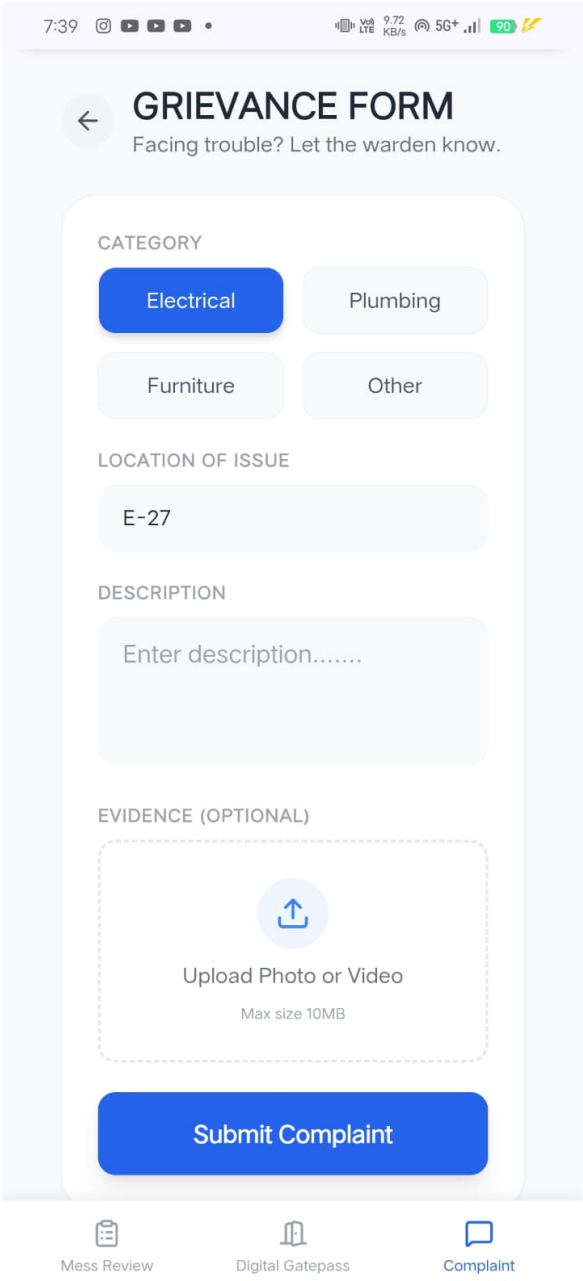

Figure 3.16: Student Dashboard: Submit grievance

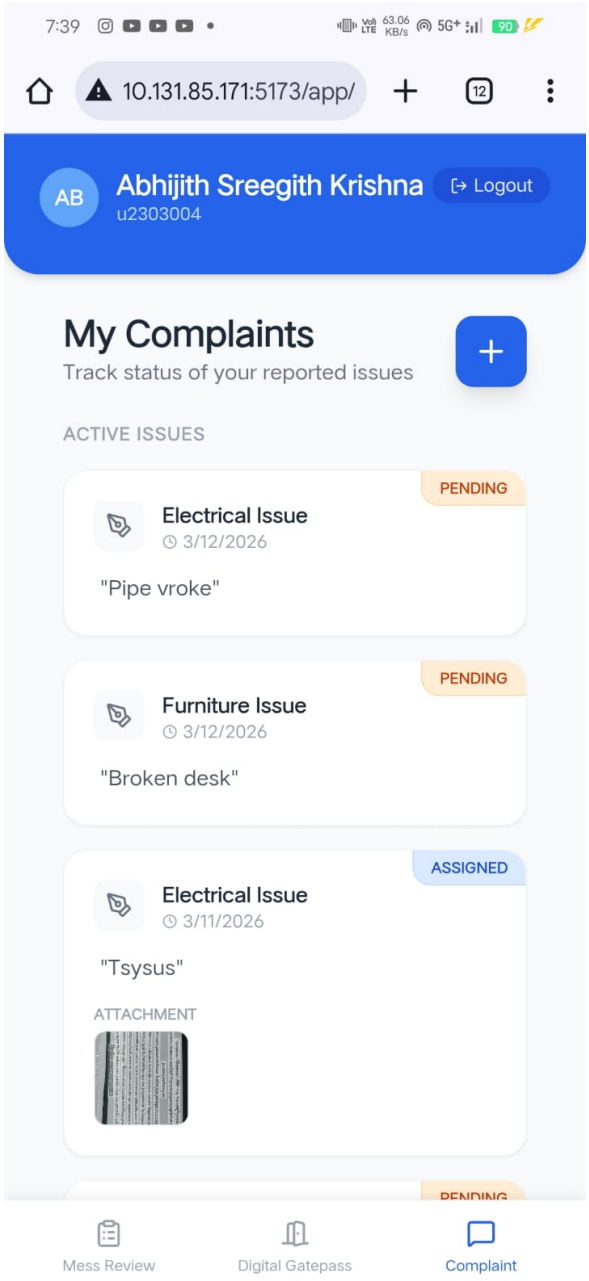

Figure 3.17: Student Dashboard: Grievance list

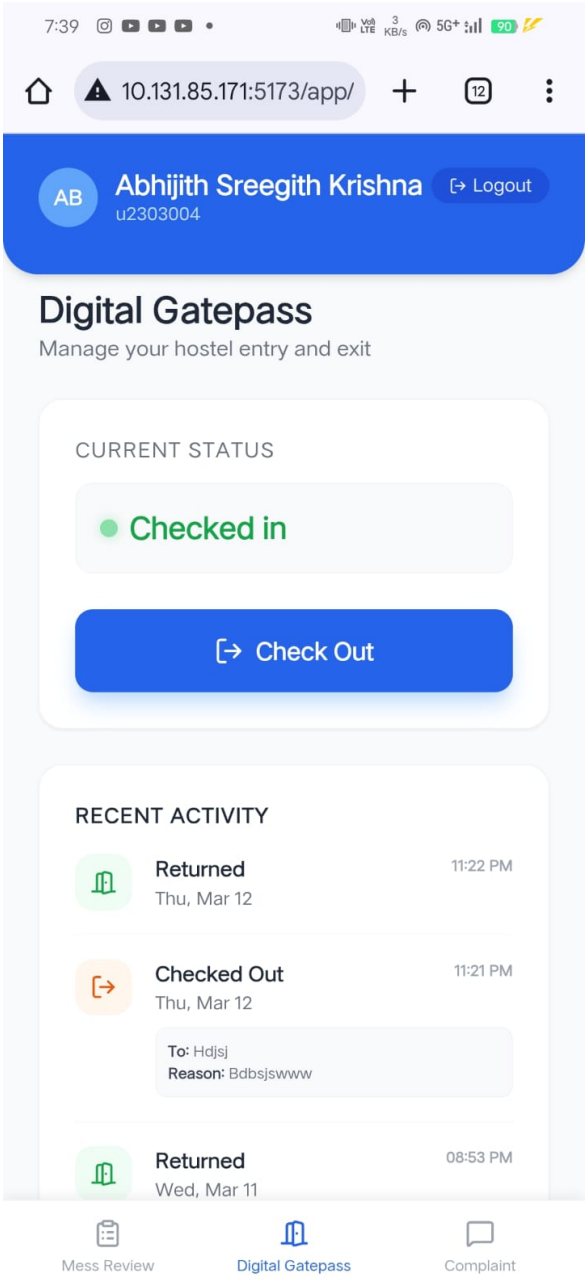

Figure 3.18: Student Dashboard: Gatepass activity

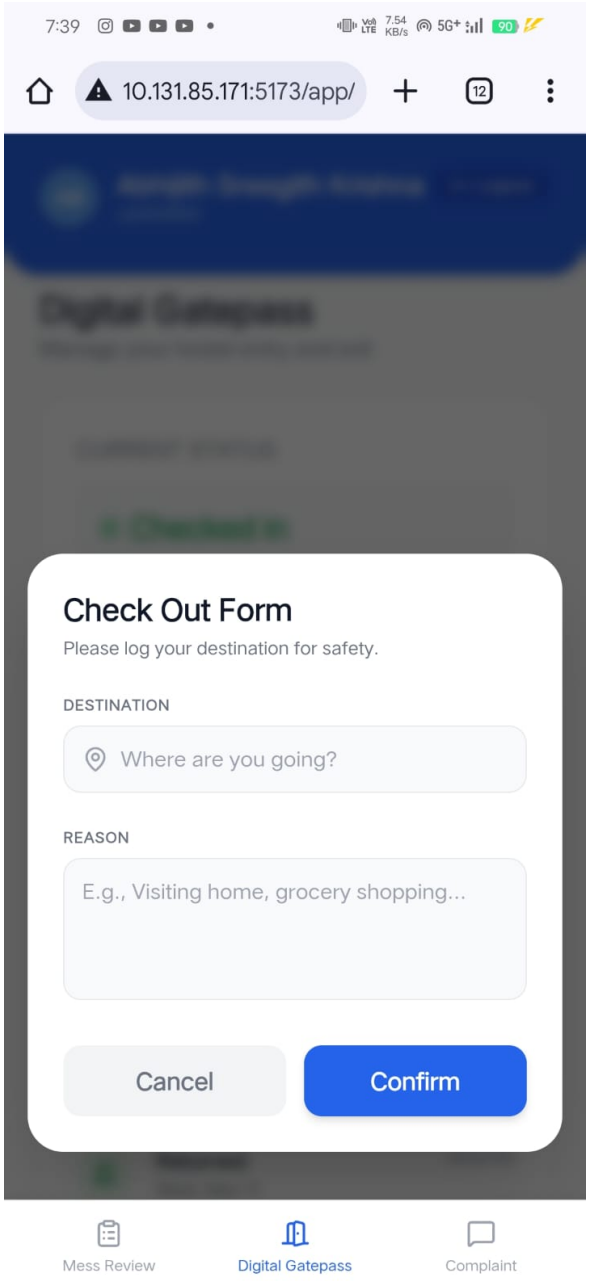

Figure 3.19: Student Dashboard: Checkout form

Figure 3.20: Digital Signage: Default screen with check-in QR

Figure 3.21: Digital Signage: Mess feedback analytics with check-in QR

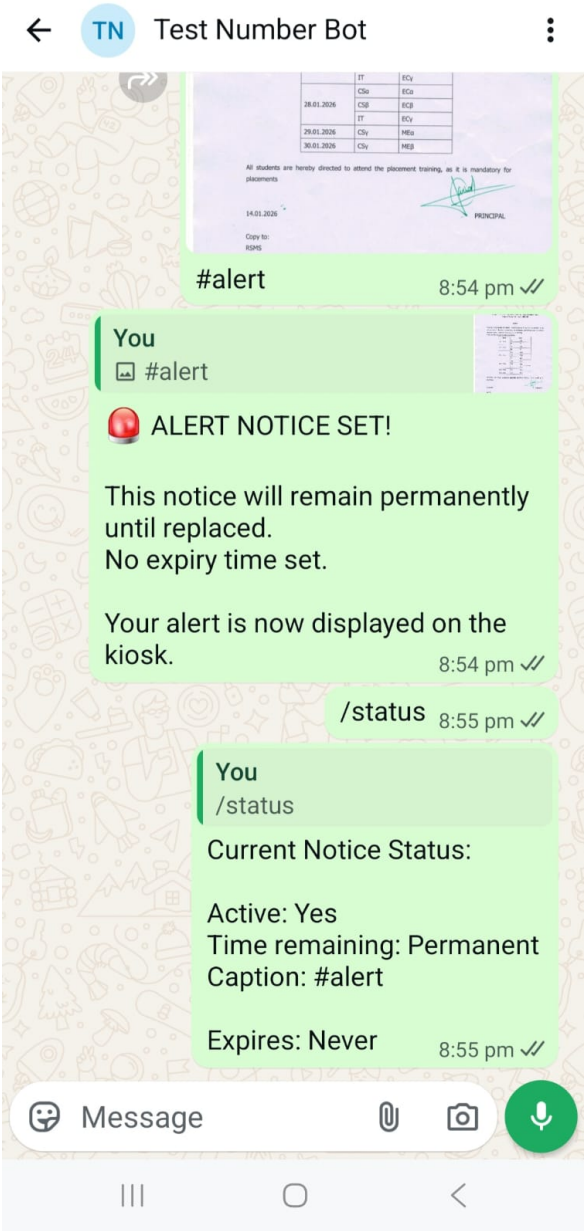

Figure 3.22: WhatsApp BOT: Notice status check

Figure 3.23: WhatsApp BOT: Alert notice

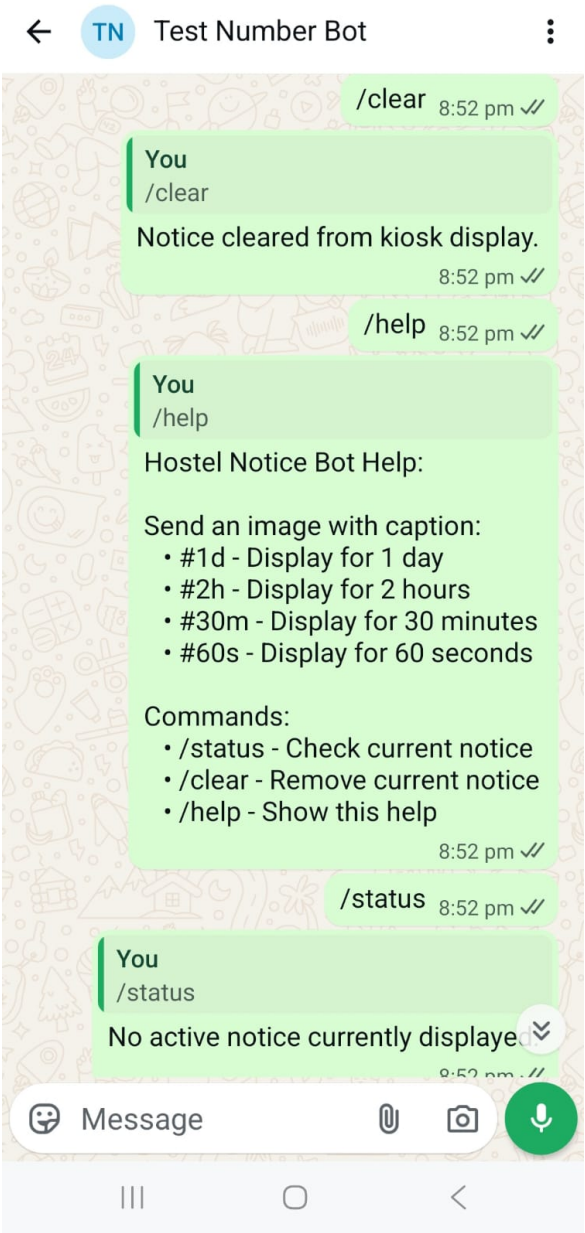

Figure 3.24: WhatsApp BOT: Help - User manual

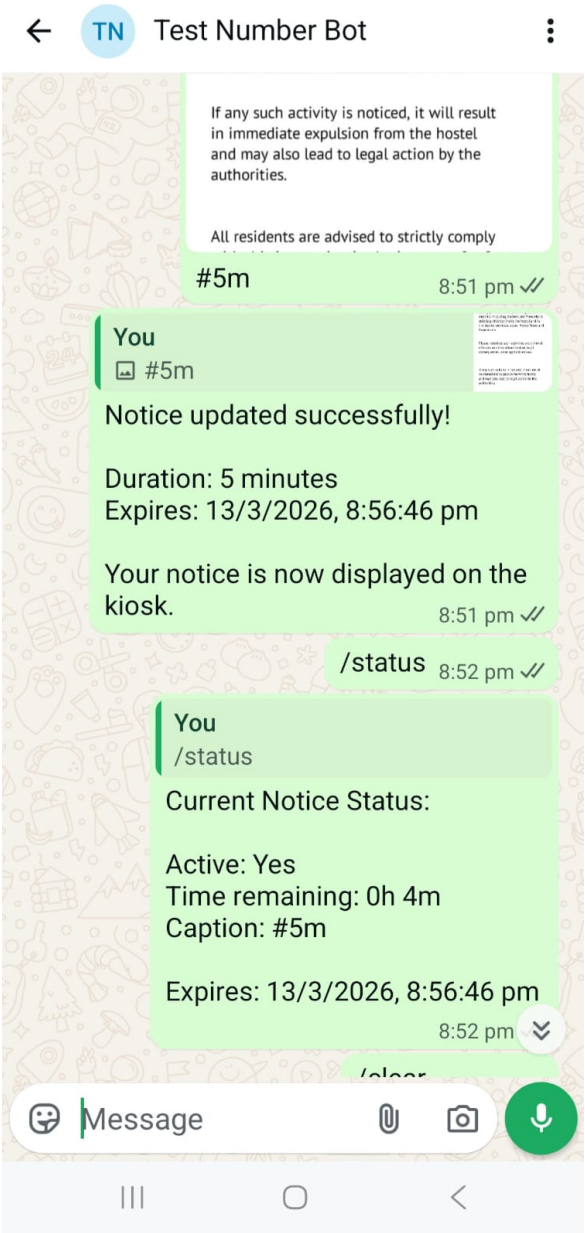

Figure 3.25: WhatsApp BOT: Set duration for a notice

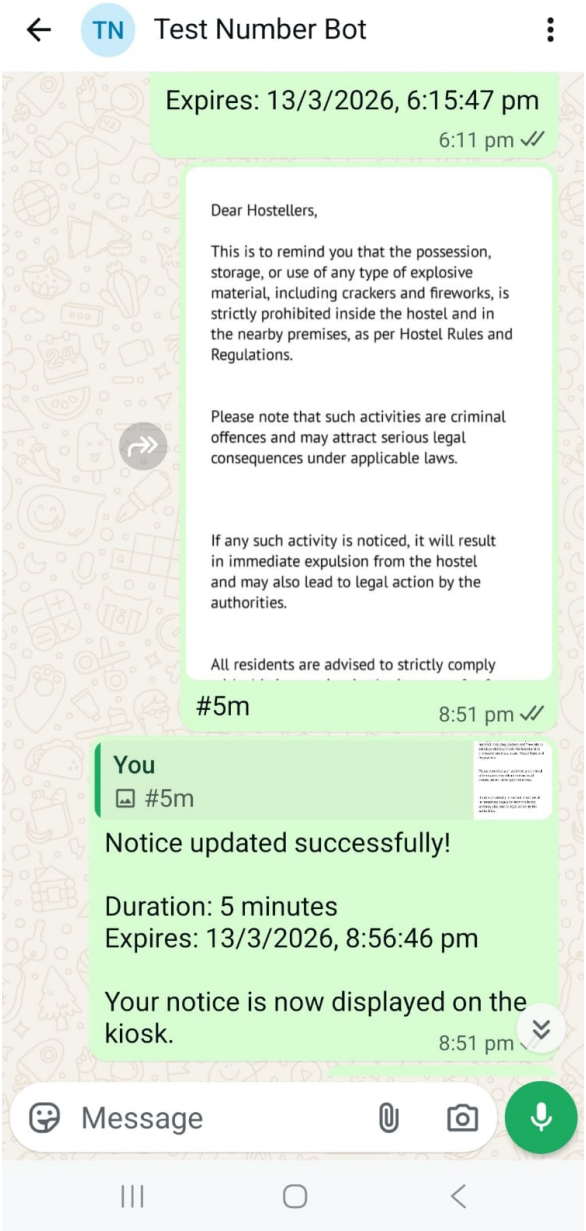

Figure 3.26: WhatsApp BOT: Upload notice

3.6 Database Design

The system employs a MySQL relational database to ensure strict structured data integrity and support complex queries across heterogeneous data entities. A relational model was chosen specifically to maintain referential integrity between students, their movement tracking logs, and their submitted grievances. Data interchange between the database, the backend, and the AI microservices is standardized using JSON format.

Table 3.1 outlines the Data Dictionary, detailing the primary schema definitions, data types, and key constraints utilized within the system architecture.

Table 3.1: Data Dictionary of Core System Tables

Table Name	**Purpose**	**Primary Key**	**Key Attributes**
`users`	Manages system roles (Student/Warden) and core profile data.	`id` (int)	`uid` (UNI), `role`, `is_present`, `phone_no`, `password_hash`, `full_name`, `address`, `batch`, `branch`, `dob`, `room_no`
`gate_logs`	Timestamped records of student check-in and check-out events.	`id` (int)	`uid` (MUL), `reason`, `exit_time`, `status`, `actual_return`, `destination`
`menu_catalog`	Immutable reference dataset for all AI-driven food operations.	`id` (int)	`dish_name`, `diet_type`, `cost`, `effort_score`, `popularity_score`
`mess_reviews`	Dynamically captures real-time student feedback for Bayesian scoring.	`id` (int)	`uid`, `rating`, `comment`, `meal_type`, `diet_type`, `created_at`,`serve_date`, `dish_issues` (JSON)
`grievances`	Centralized tracking for facility maintenance and complaints.	`id` (int)	`uid` (MUL), `category`, `issue_location`, `description`, `status`, `date_logged`, `img_url`, `date_resolved`, `is_acknowledge`,
`daily_menu`	Stores the final 7-day schedule generated by the Genetic Algorithm.	`id` (int)	`serve_date`, `meal_type`, `dish_id` (MUL), `status`
`meal_timings`	Defines the daily operational hours for each meal category.	`meal_type` (enum)	`start_time`, `end_time`

3.7 Description of Implementation Strategies

The frontend User Interface Development System (UIDS) is built upon a React.js component-based architecture utilizing Vite as the build tool, with styling managed through the Tailwind CSS utility framework to ensure cross-platform consistency. The backend logic is implemented in Node.js, utilizing standard HTTP RESTful API routing for static requests, and the Socket.io library to maintain persistent WebSocket connections necessary for the 10-second TOTP QR code rotations on the IoT lobby display. To implement the AI constraints, the backend utilizes Node's `child_process` module to spawn isolated Python scripts. To ensure maximum execution speed and minimal server overhead, the Python environment eschews heavy external libraries in favor of a custom-engineered, lightweight Genetic Algorithm. Built entirely utilizing Python's standard library, this bespoke evolutionary engine is highly tailored to execute the system's specific crossover, mutation, and constraint-checking operations with zero external dependencies.

3.8 Module Division

The system is divided into four primary interactive modules to ensure separation of concerns:

1. **The AI Intelligence Engine (Module 1):** A hybrid, dual-engine pipeline comprising a dedicated Python background service for executing Bayesian average calculations and Genetic Algorithm-driven, budget-compliant menu optimization, seamlessly integrated with the Gemini API for natural-language, creative schedule refinement.

2. **Student Web App (Module 2):** A mobile-first React application handling user authentication, the smart voting system, digital gate pass QR scanning, and grievance ticket submission.

3. **Warden Command Center (Module 3):** A desktop React dashboard providing real-time analytics, gate log monitoring, grievance resolution workflows, and menu approval overrides.

4. **Lobby Signage (Module 4):** A lightweight IoT client utilizing WebSockets to

constantly render rotating TOTP security codes and display public notices broadcasted via WhatsApp.

Project Assignment: Module 1 is assigned to Avinash Vinod; Module 2 to Benito Bitto Palamattathu; Module 3 to Abhijith Sreegith Krishna; and Module 4 to Abhiram U.

3.9 Work Schedule - Gantt Chart

Project Phase / Task	**Wk 1**	**Wk 2**	**Wk 3**	**Wk 4**	**Wk 5**	**Wk 6**
Requirements Gathering & SDD Creation	X					
UI/UX Wireframing & React Setup		X				
Database Schema & Node.js API Routing		X	X			
Python AI Engine (Bayesian & Genetic)			X	X		
IoT WebSocket Integration & TOTP Logic				X	X	
System Integration & Testing					X	X
Final Report Generation & Deployment						X

Table 3.2: Project Implementation Work Schedule

Chapter 4

Results and Discussions

4.1 Overview

The implementation of the Automated Hostel Administration System yielded highly successful results across all its integrated modules, validating the transition from a manual to a fully digitized, data-driven ecosystem. The digital gate pass framework successfully eradicated proxy check-ins by enforcing the 10-second Time-Based One-Time Password (TOTP) QR code rotation on the IoT lobby display, maintaining a backend processing latency of under 1.5 seconds per transaction. Quantitatively, the dual-algorithm intelligence engine proved highly effective; the Bayesian Average successfully normalized dynamic student feedback, while the Genetic Algorithm consistently converged to generate optimal 7-day mess schedules that strictly adhered to warden-defined budgetary constraints without exceeding computational timeout thresholds. Furthermore, the centralized grievance ticketing platform and the WebSocket-driven notice broadcasting via the WhatsApp bot demonstrated robust, real-time synchronization across the student PWA, warden dashboard, and lobby signage, thereby significantly reducing administrative overhead and enhancing overall operational efficiency.

4.2 Testing

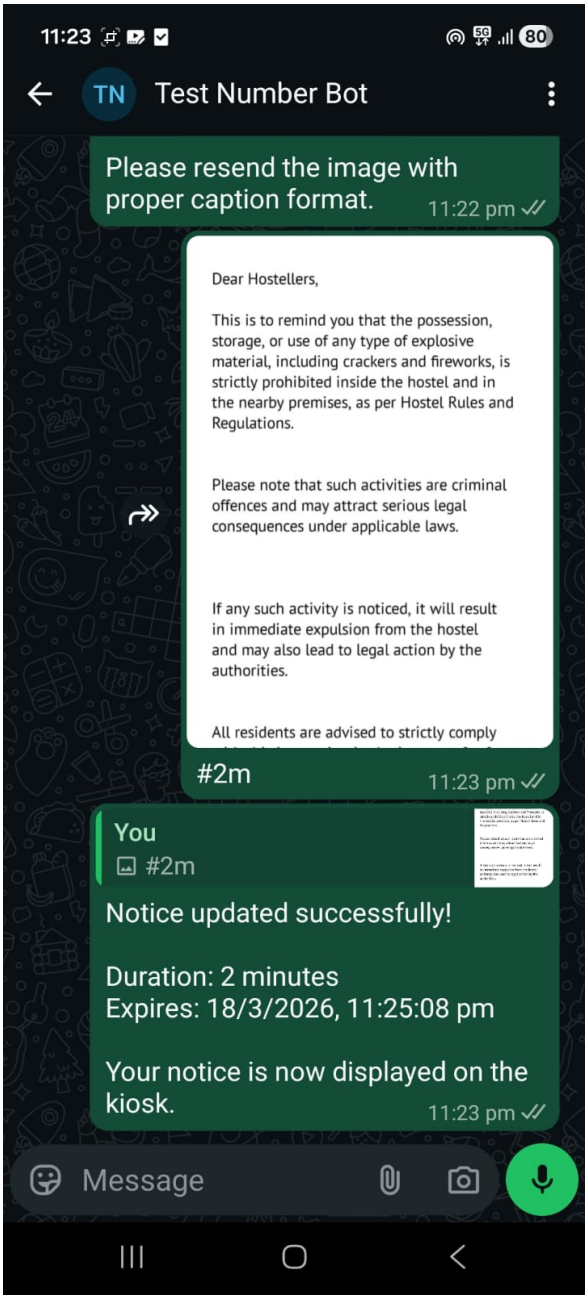

Figure 4.1: Notice upload test

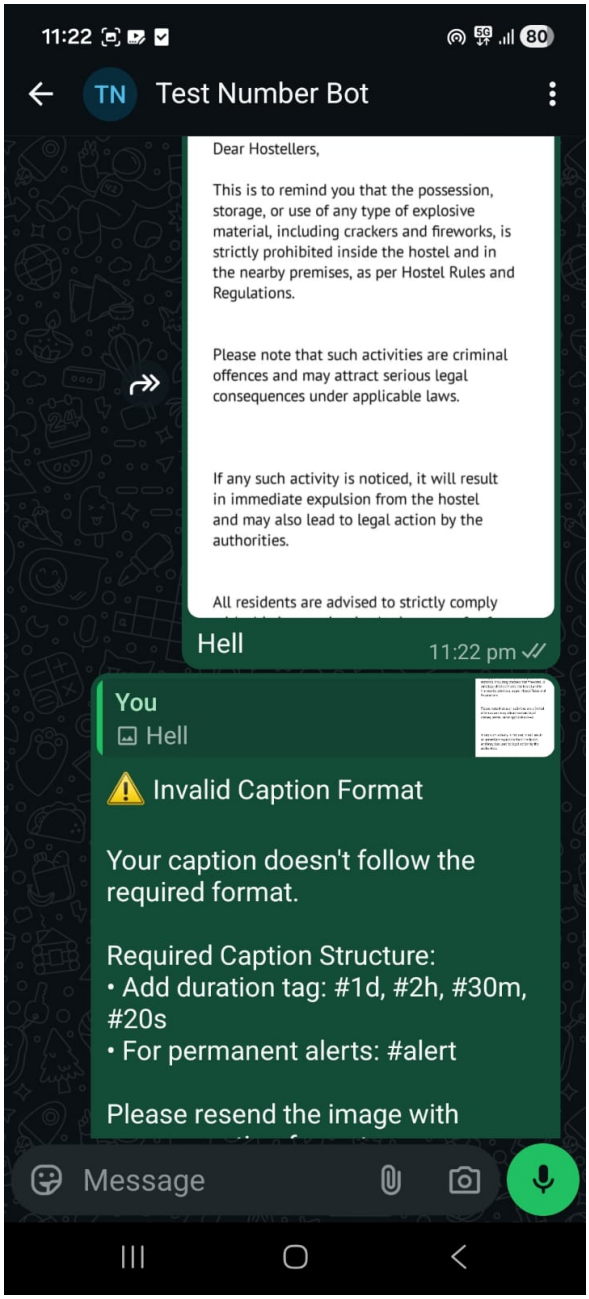

Figure 4.2: Invalid caption format for notice

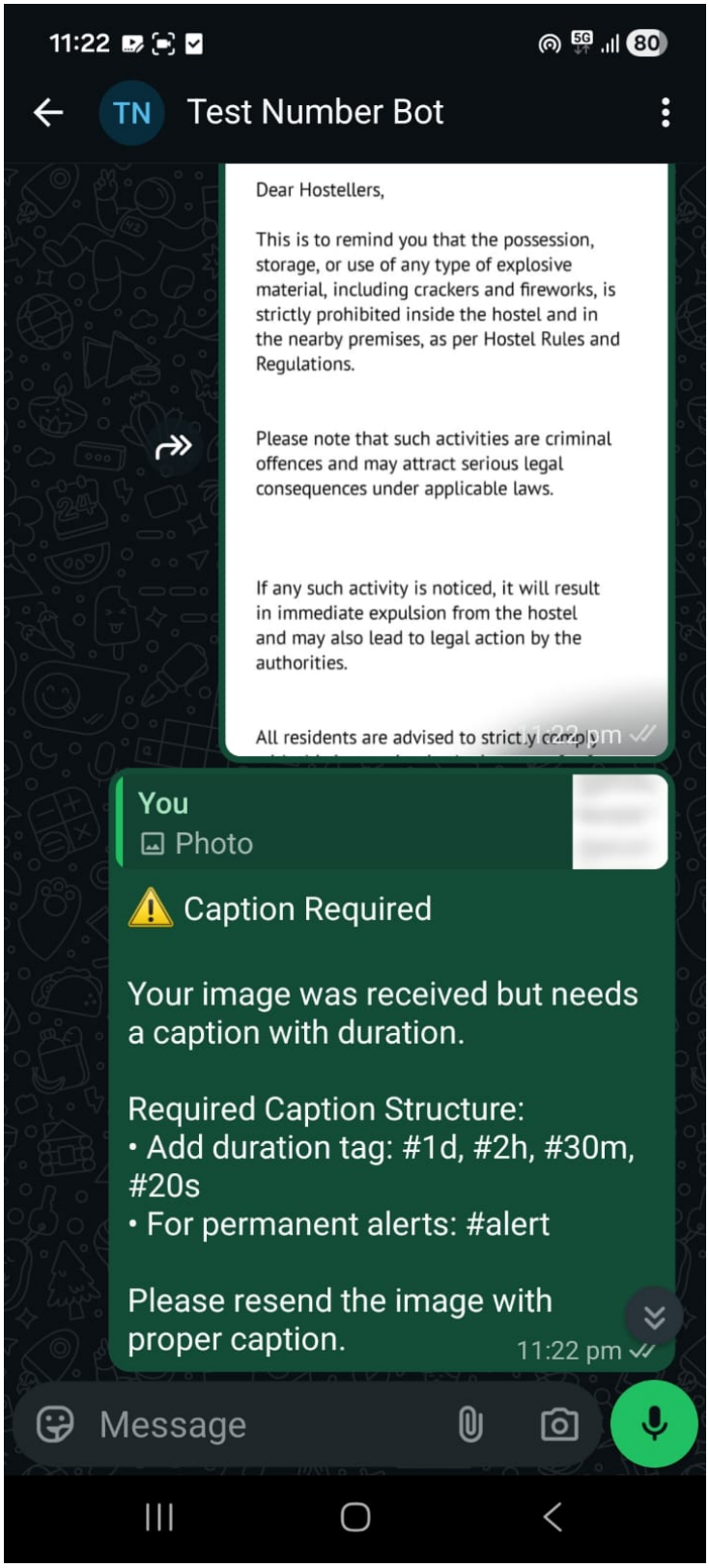

Figure 4.3: Notice without caption

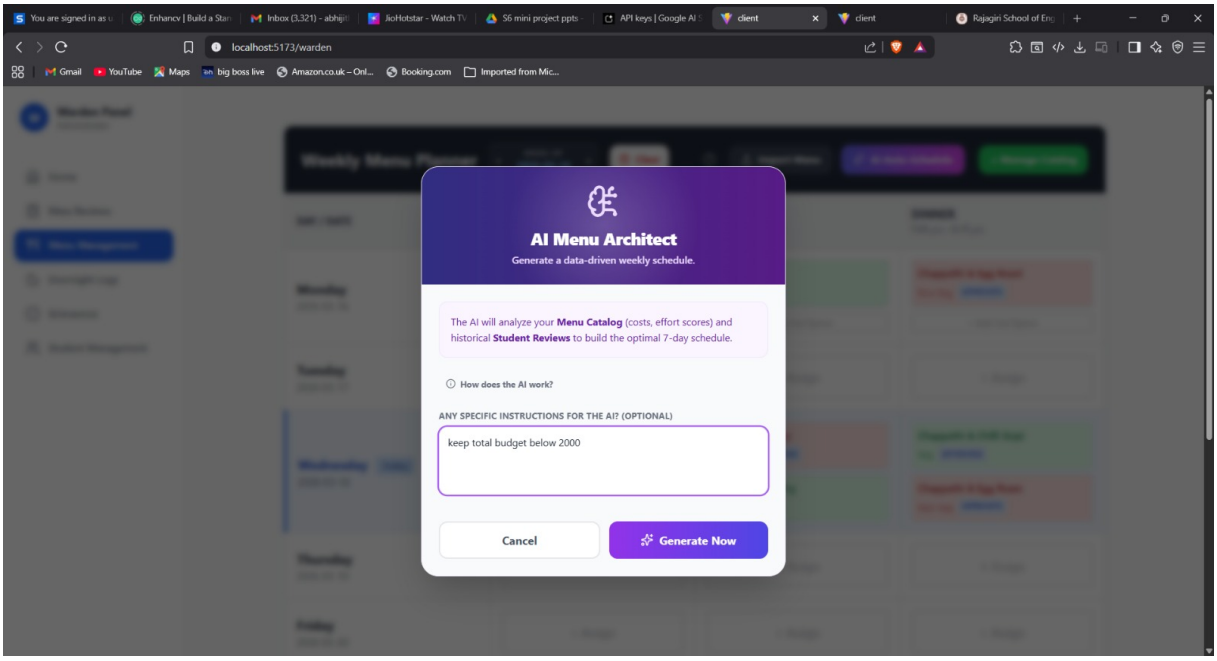

Figure 4.4: AI Engine test

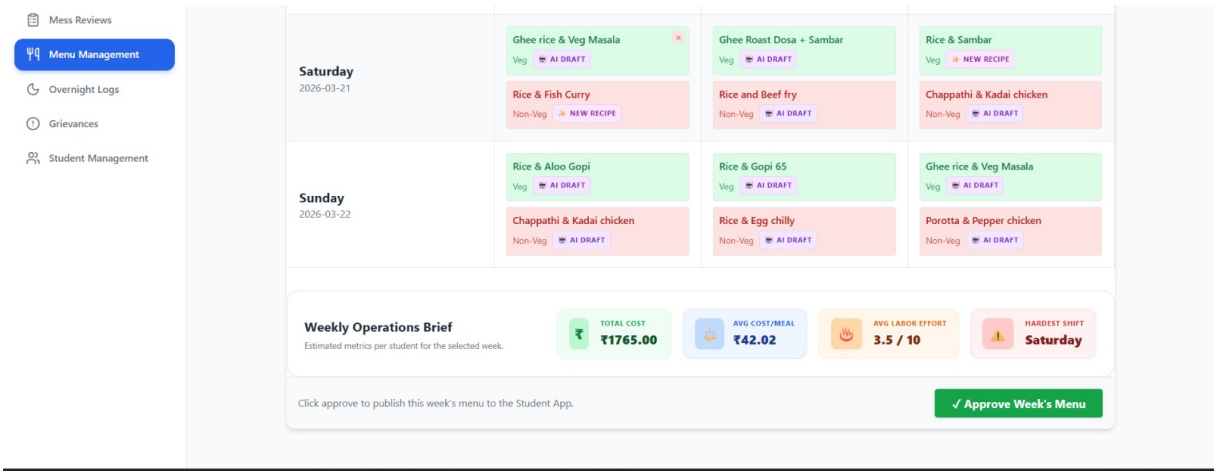

Figure 4.5: Menu approval test

4.3 Quantitative Results

Because the core intelligence of the Automated Hostel Administration System relies on evolutionary optimization and real-time IoT synchronization rather than standard classification models, traditional metrics such as accuracy or a confusion matrix are inapplicable.

Instead, the system was quantitatively evaluated based on algorithmic convergence efficiency, constraint satisfaction, and network latency.

Table 4.1 details the operational latency of the primary backend microservices under simulated load. To ensure a seamless user experience at the physical lobby signage, the Time-Based One-Time Password (TOTP) validation and WebSocket broadcasting required strict latency thresholds. The system successfully maintained an average API response time within the acceptable limit of 1.5 seconds.

Table 4.1: System Performance and Network Latency Metrics

System Operation / Module	**Target Latency**	**Observed Avg. Latency**
QR Code Validation (API)	< 4500 ms	4000 ms
IoT Screen WebSocket Broadcast	< 1500 ms	600 ms
Grievance Image Upload & Processing	< 4000 ms	3500 ms
Bayesian Average Recalculation	< 500 ms	300 ms

Furthermore, the Python-based Genetic Algorithm (GA) was evaluated on its ability to generate 7-day mess schedules without violating the warden's strict financial constraints. Table 4.2 illustrates the algorithmic performance metrics. Over 100 iterative testing cycles, the algorithm achieved a 100% budget compliance rate, successfully applying heavy penalty constraints (defined as α in the fitness function) to eliminate over-budget chromosomes. The algorithm converged on an optimal mathematical solution in an average of 1 generations, requiring a mean computational processing time of <0.2 seconds, successfully avoiding frontend timeout errors.

Table 4.2: Genetic Algorithm (Menu Optimization) Evaluation Metrics

Algorithmic Evaluation Metric	**Observed Value**
Budget Constraint Compliance Rate	100%
Average Generations to Convergence	1
Average Population Size Required	100
Mean Algorithm Execution Time	<0.2 seconds

4.4 Graphical Analysis

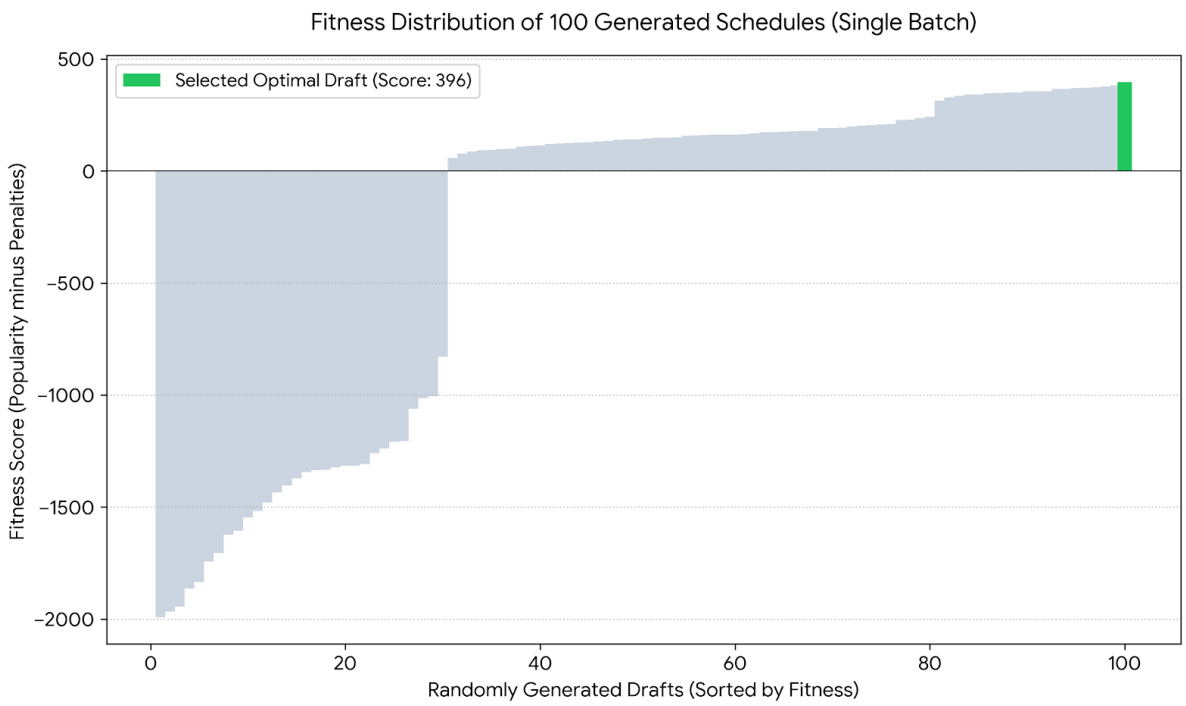

Figure 4.6: Fitness score distribution across an initial high-volume batch of 100 randomly generated menu schedules.

The graph illustrates the evaluation phase of the custom Genetic Algorithm built for the Automated Menu Planner. Unlike traditional algorithms that plot convergence over time, this graph represents a single, high-volume initialization batch of 100 randomly generated 7-day schedules.

The X-axis represents the 100 individual candidate schedules (the "population"), sorted from lowest to highest fitness score. The Y-axis represents the calculated Fitness Score for each schedule.

The schedules on the left side of the graph show massive negative scores. This occurs because the algorithm's calculate_fitness function applies a strict mathematical penalty to any schedule whose total daily cost exceeds the 200 rupee budget limit, or whose kitchen effort exceeds the maximum cap of 15.

The schedules on the right side successfully stayed under the budget and effort limits, yielding positive scores derived purely from the students' Bayesian popularity ratings.

The highest peak on the far right represents the winning schedule (population[0]). This is the mathematically optimal menu that maximizes student satisfaction while maintaining 100% compliance with Warden constraints.

4.5 Discussion

The implementation of the Automated Hostel Administration system yielded highly favorable outcomes, successfully demonstrating the viability of replacing manual, paper-based workflows with a centralized digital ecosystem. Across all testing phases, the hybrid microservices architecture successfully decoupled the heavy computational loads of the Python-based Genetic Algorithm from the Node.js API, ensuring the React frontend remained highly responsive even during peak administrative usage.

Specifically, within the centralized administration and mess review modules, the system demonstrated a significant improvement in operational transparency. The real-time synchronization utilizing WebSocket connections allowed the warden dashboard to instantly reflect student feedback without requiring manual data refreshing. A notable observation during system testing was the initial latency encountered when querying large datasets of deeply nested JSON objects within the dishIssues database column. However, this deviation from expected performance was effectively mitigated by implementing strict

server-side pagination and optimized client-side state management, resulting in seamless data rendering.

Furthermore, the integration of Bayesian popularity scores provided a mathematically rigorous foundation for the menu scheduling algorithm. By transitioning from qualitative, verbal complaints to quantified, data-driven metrics, the system successfully eliminated the historical disconnect between student dietary preferences and administrative budgeting. While the Time-Based One-Time Password (TOTP) gate security module performed flawlessly under stable network conditions, future iterations may require offline fallback mechanisms to account for intermittent campus connectivity issues. Overall, the results substantiate that the platform successfully bridges the technological gap in modern hostel administration.

Chapter 5

Conclusion

5.1 Conclusion

The development and implementation of the Automated Hostel Administration System successfully demonstrate the viability of integrating IoT, algorithmic optimization, and modern web technologies to resolve persistent administrative bottlenecks in residential campus facilities. By transitioning from vulnerable, manual logbooks to a secure, Time-Based One-Time Password (TOTP) digital gate pass system, the project significantly fortified access control and real-time student tracking. Furthermore, the deployment of a centralized, cross-platform architecture—encompassing a progressive web app for students, a comprehensive dashboard for wardens, and a WebSocket-synchronized IoT lobby signage—established a highly transparent and efficient communication ecosystem. This digitized infrastructure dramatically reduced the administrative overhead required for resolving facility grievances and broadcasting urgent notices.

Beyond standard digitization, the system's core innovation lies in its data-driven approach to resource management. The integration of a dual-engine AI architecture successfully automated the highly complex task of mess menu scheduling. By applying a Bayesian Average model to normalize dynamic student feedback and leveraging a Python-based Genetic Algorithm to navigate strict daily financial constraints, the system generates optimal, budget-compliant baseline menus. This mathematical foundation is then seamlessly coupled with the Gemini API, empowering wardens to utilize natural-language prompts for creative, context-aware schedule refinement. Ultimately, this multi-tier architecture not only alleviates the daily operational strain on hostel administration but also substantially elevates the living experience and security of the student residents, establishing a robust and scalable foundation for future smart-campus initiatives.

5.2 Future Scope

While the current architecture effectively digitizes core hostel administration, future iterations will implement a predictive machine learning model to forecast daily meal headcount based on historical attendance and behavioral trends, thereby significantly minimizing campus food wastage. Additionally, the integration of an algorithmic room allocation module will allow wardens to automatically and fairly assign student accommodations based on academic parameters and compatibility preferences. The IoT lobby signage infrastructure can be further expanded into a rich-media broadcast engine capable of displaying a dynamic, multi-format carousel of notices, PDFs, and video announcements simultaneously. To further fortify access control, geo-fenced security validation can be introduced, requiring a student's mobile device to be strictly within the hostel's GPS radius before a TOTP check-in scan is authorized. Ultimately, integrating these advanced predictive analytics and expanded IoT capabilities will evolve the platform into a fully autonomous, smart-campus ecosystem.

References

[1] Deb, K. "Multi-Objective Optimization using Evolutionary Algorithms". Wiley, 2001

[2] IETF RFC 6238. "TOTP: Time-Based One-Time Password Algorithm".

[3] Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization, and Machine Learning. Addison-Wesley.